1. Content analysis: Analyze the content of social media data to determine what topics are being given data set using topic modelling, keyword extractor

```
import pandas as pd
import nltk
import gensim
from gensim import corpora
from gensim.models import LdaModel
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from collections import Counter
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
nltk.download('stopwords')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to C:\Users\Om
[nltk_data]     Shete\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to C:\Users\Om
[nltk_data]     Shete\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
True
```

```
df = pd.read_csv('CSV/social_media_data.csv')
print(df.head())
```

```
   id                                          text
0   1  Just had a great meal at the new restaurant do...
1   2  I love going to the beach with my family in th...
2   3  Can't wait to see the new movie coming out thi...
3   4  Feeling stressed out at work today, need a vac...
4   5         I can't believe how fast time is flying by
```

```
stop_words = set(stopwords.words('english'))
print("Stopwords: ")
print(stop_words)
lemmatizer = WordNetLemmatizer()
```

```
Stopwords:
{'aren', "it's", 'further', 'on', 'ourselves', 'under', 'himself', 'to', 'do', 'where', 'no', 'yourselves', 'needn', 'having', 'she
```

```
def pre_process(text):
    tokens = text.lower().split()
    lemma = [ lemmatizer.lemmatize(token) for token in tokens if token not in stop_words]
    return lemma
```

```
df['tokens'] = df['text'].apply(pre_process)
print(df.head())
```

```
   id  ...                                          tokens
0   1  ...          [great, meal, new, restaurant, downtown!]
1   2  ...              [love, going, beach, family, summer]
2   3  ...    [can't, wait, see, new, movie, coming, weekend]
3   4  ...  [feeling, stressed, work, today,, need, vacation]
4   5  ...               [can't, believe, fast, time, flying]

[5 rows x 3 columns]
```

```
# Topic Modeling with lda

dictionary = corpora.Dictionary(df['tokens'])
corpus = [dictionary.doc2bow(token) for token in df['tokens']]

num_topics = 5
lda_model = LdaModel(num_topics=num_topics, corpus=corpus, id2word=dictionary, passes=20)

print("Topics: ")
for topic_id in lda_model.print_topics():
    print(f"Topic: {topic_id}")
```

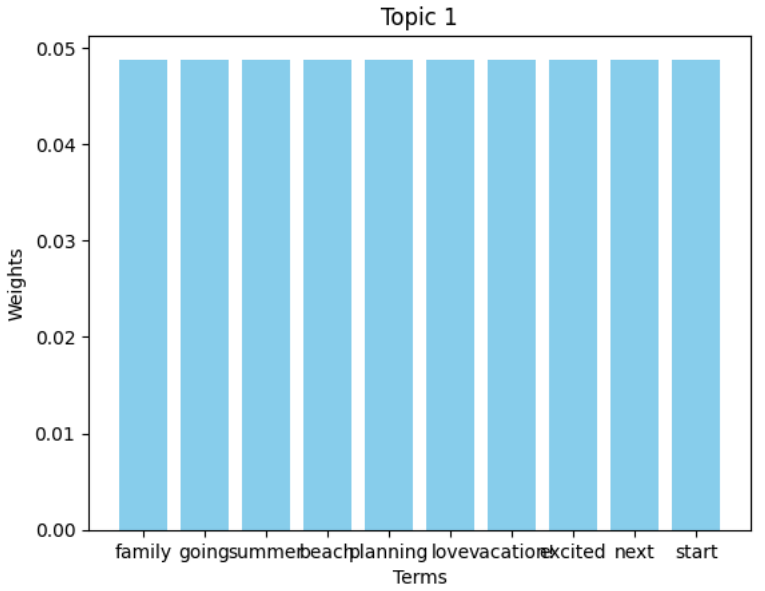
```
Topics:
Topic: (0, '0.049*"family" + 0.049*"going" + 0.049*"summer" + 0.049*"beach" + 0.049*"planning" + 0.049*"love" + 0.049*"vacation!" +
Topic: (1, '0.042*"believe" + 0.042*"live" + 0.042*"even" + 0.042*"amazing," + 0.042*"sounded" + 0.042*"band" + 0.042*"concert" + 0
Topic: (2, '0.062*"day" + 0.043*"feeling" + 0.043*"can\'t" + 0.043*"movie" + 0.043*"need" + 0.043*"today," + 0.023*"eating" + 0.023*
Topic: (3, '0.070*"great" + 0.038*"new" + 0.038*"highly" + 0.038*"finished" + 0.038*"reading" + 0.038*"it!" + 0.038*"book," + 0.038*
```

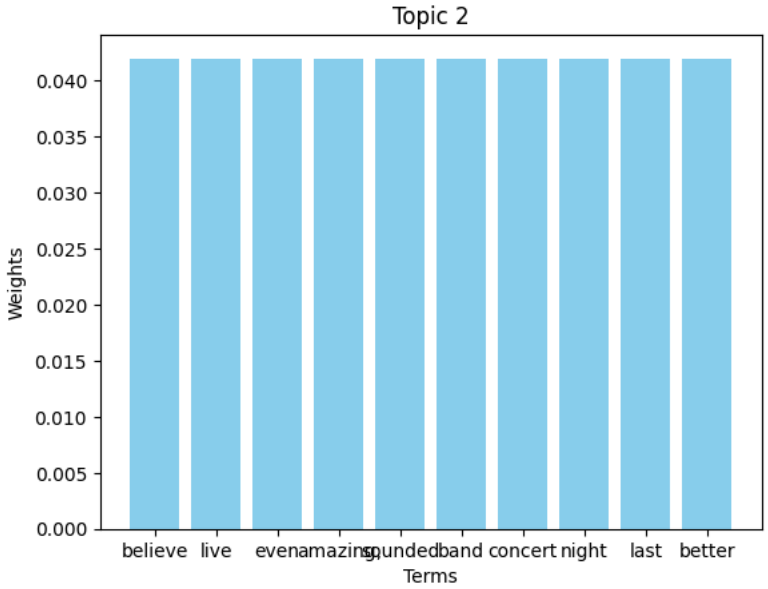
```
Topic: (4, '0.045*"new" + 0.045*"start" + 0.045*"next" + 0.045*"excited" + 0.045*"incredible" + 0.045*"mountains," + 0.045*"hiking"
```

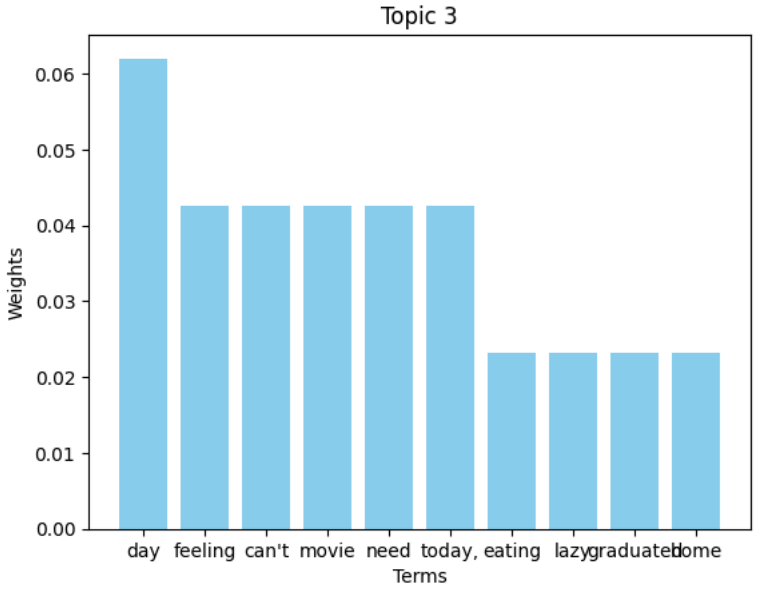
```
for idx in range(num_topics):
    topic_info = lda_model.print_topic(idx, topn=10)
    word_and_weights = topic_info.split(" + ")
    print(word_and_weights)

    for item in word_and_weights:
        word, weight = item.split("*")
        print(f" - {word.strip()} (Weight: {weight.strip()})")
```

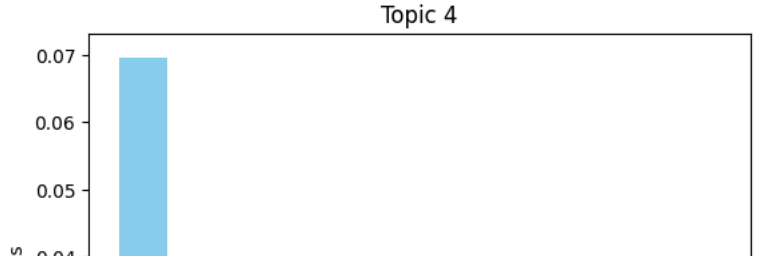
```
['0.049*"family"', '0.049*"going"', '0.049*"summer"', '0.049*"beach"', '0.049*"planning"', '0.049*"love"', '0.049*"vacation!"', '0.0
 - 0.049 (Weight: "family")
 - 0.049 (Weight: "going")
 - 0.049 (Weight: "summer")
 - 0.049 (Weight: "beach")
 - 0.049 (Weight: "planning")
 - 0.049 (Weight: "love")
 - 0.049 (Weight: "vacation!")
 - 0.049 (Weight: "excited")
 - 0.049 (Weight: "next")
 - 0.049 (Weight: "start")
['0.042*"believe"', '0.042*"live"', '0.042*"even"', '0.042*"amazing,"', '0.042*"sounded"', '0.042*"band"', '0.042*"concert"', '0.042
 - 0.042 (Weight: "believe")
 - 0.042 (Weight: "live")
 - 0.042 (Weight: "even")
 - 0.042 (Weight: "amazing,")
 - 0.042 (Weight: "sounded")
 - 0.042 (Weight: "band")
 - 0.042 (Weight: "concert")
 - 0.042 (Weight: "night")
 - 0.042 (Weight: "last")
 - 0.042 (Weight: "better")
['0.062*"day"', '0.043*"feeling"', '0.043*"can\'t"', '0.043*"movie"', '0.043*"need"', '0.043*"today,"', '0.023*"eating"', '0.023*"la
 - 0.062 (Weight: "day")
 - 0.043 (Weight: "feeling")
 - 0.043 (Weight: "can't")
 - 0.043 (Weight: "movie")
 - 0.043 (Weight: "need")
 - 0.043 (Weight: "today,")
 - 0.023 (Weight: "eating")
 - 0.023 (Weight: "lazy")
 - 0.023 (Weight: "graduated")
 - 0.023 (Weight: "home")
['0.070*"great"', '0.038*"new"', '0.038*"highly"', '0.038*"finished"', '0.038*"reading"', '0.038*"it!"', '0.038*"book,"', '0.038*"re
 - 0.070 (Weight: "great")
 - 0.038 (Weight: "new")
 - 0.038 (Weight: "highly")
 - 0.038 (Weight: "finished")
 - 0.038 (Weight: "reading")
 - 0.038 (Weight: "it!")
 - 0.038 (Weight: "book,")
 - 0.038 (Weight: "recommend")
 - 0.038 (Weight: "restaurant")
 - 0.038 (Weight: "downtown!")
['0.045*"new"', '0.045*"start"', '0.045*"next"', '0.045*"excited"', '0.045*"incredible"', '0.045*"mountains,"', '0.045*"hiking"', '0
 - 0.045 (Weight: "new")
 - 0.045 (Weight: "start")
 - 0.045 (Weight: "next")
 - 0.045 (Weight: "excited")
 - 0.045 (Weight: "incredible")
 - 0.045 (Weight: "mountains,")
 - 0.045 (Weight: "hiking")
 - 0.045 (Weight: "view")
 - 0.045 (Weight: "afternoon")
 - 0.045 (Weight: "spent")
```

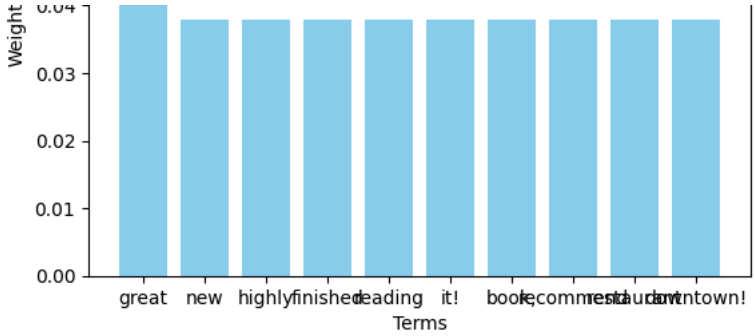
```
for idx, topic in enumerate(lda_model.show_topics(num_topics, formatted=False)):
    topic_term = [term[0] for term in topic[1]]
    term_weight = [term[1] for term in topic[1]]

    plt.figure()
    plt.bar(topic_term, term_weight, color='skyblue')
    plt.title(f"Topic {idx + 1}")
    plt.xlabel("Terms")
    plt.ylabel("Weights")
    plt.show()
```

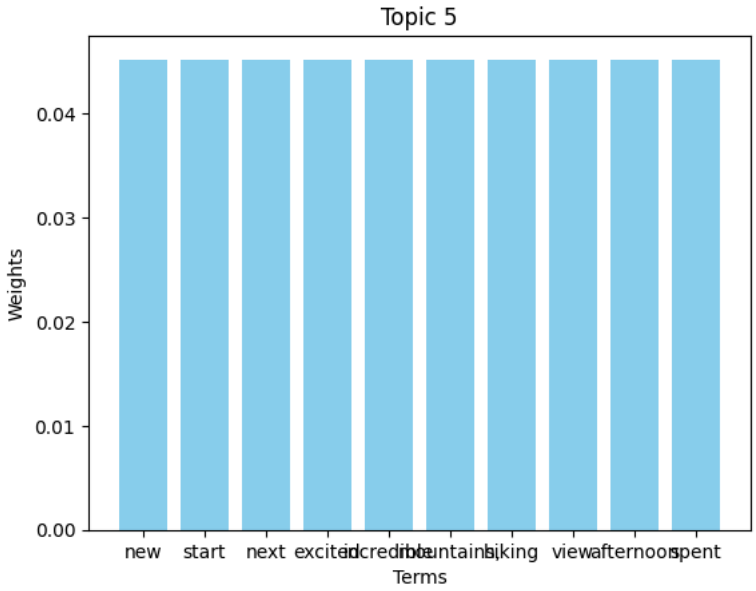

Topic 1
Weights
0.05
0.04
0.03
0.02
0.01
0.00
family
going
summer
beach
planning
love
vacation
excited
next
start
Terms


Topic 2
Weights
0.040
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000
believe
live
even
amazing
sounded
band
concert
night
last
better
Terms


Topic 3
Weights
0.06
0.05
0.04
0.03
0.02
0.01
0.00
day
feeling
can't
movie
need
today,
eating
lazy
graduated
home
Terms


Topic 4
0.07
0.06
0.05






```
# keyword extraction
all_tokens = [ token for tokens in df['tokens'] for token in tokens]
keyword_ctr = Counter(all_tokens)
print("\nKeywords Counts: \n", keyword_ctr)

tfidf = TfidfVectorizer(max_features=20, stop_words='english')
x = tfidf.fit_transform(df['text'])

tfidf_keyword = tfidf.get_feature_names_out()
print("\nTop TF-IDF Keywords: ", tfidf_keyword)
```

```
Keywords Counts:
 Counter({'new': 3, "can't": 3, 'feeling': 3, 'day': 3, 'great': 2, 'family': 2, 'movie': 2, 'today,': 2, 'need': 2, 'believe': 2,

Top TF-IDF Keywords:  ['believe' 'day' 'excited' 'family' 'feeling' 'great' 'just' 'movies'
 'need' 'new' 'night' 'park' 'perfect' 'planning' 'popcorn' 'reading'
 'recommend' 'start' 'today' 'vacation']
```

2. Location analysis: Analyze the location data associated with tweets to understand where particular location are most prevalent.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('location.csv')
df.head()
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweets	Likes	Country	Year	Month	Day	H
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15.0	30.0	USA	2023	1	15	
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5.0	10.0	Canada	2023	1	15	
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20.0	40.0	USA	2023	1	15	
3	3	3	Excited about the upcoming weekend	Positive	2023-01-15	AdventureX	Facebook	#Travel #Adventure	8.0	15.0	UK	2023	1	15	

Next steps: Generate code with df | View recommended plots | New interactive sheet

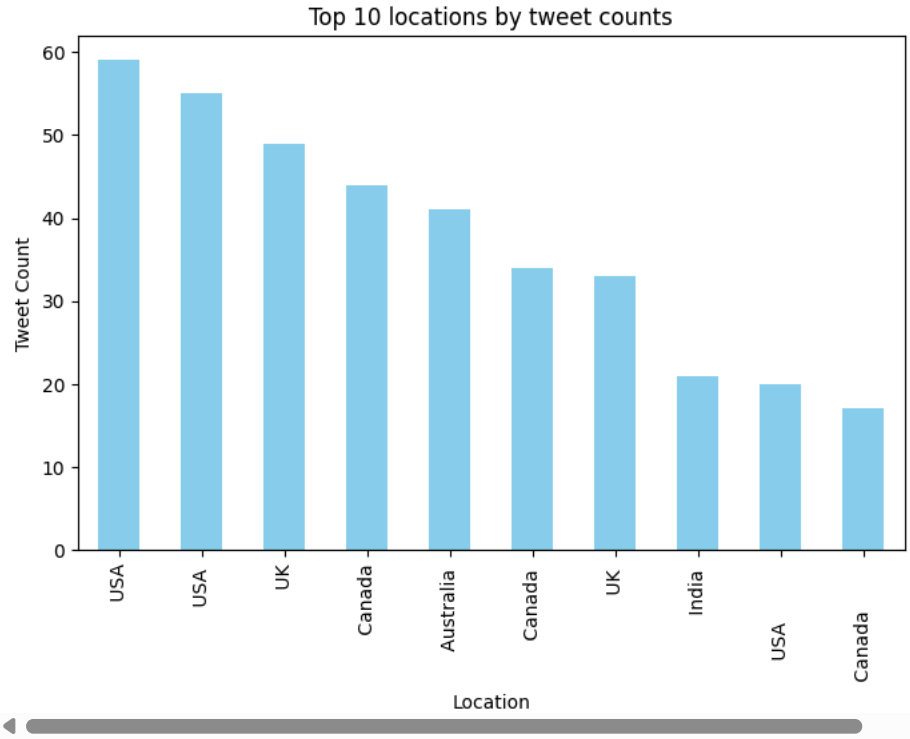
```
location_ctr = df['Country'].value_counts()
print(location_ctr)
```

```
Country
USA          59
USA          55
UK           49
Canada       44
Australia    41
             ..
Ireland       1
Scotland      1
Kenya         1
Jamaica       1
Thailand      1
Name: count, Length: 115, dtype: int64
```

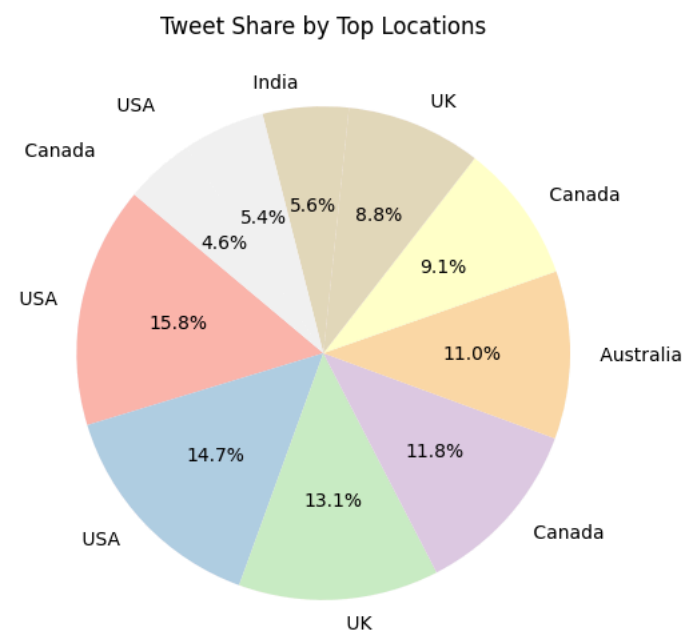
```
top_locations = location_ctr.head(10)
print(top_locations)
top_location_names = top_locations.index.tolist()
df_top = df[df['Country'].isin(top_location_names)]
```

```
Country
USA              59
USA              55
UK               49
Canada           44
Australia        41
Canada           34
UK               33
India            21
USA              20
Canada           17
Name: count, dtype: int64
```

```
top_locations.plot(kind='bar', figsize=(8,5), color='skyblue')
plt.title('Top 10 locations by tweet counts')
plt.xlabel("Location")
plt.ylabel("Tweet Count")
plt.show()
```

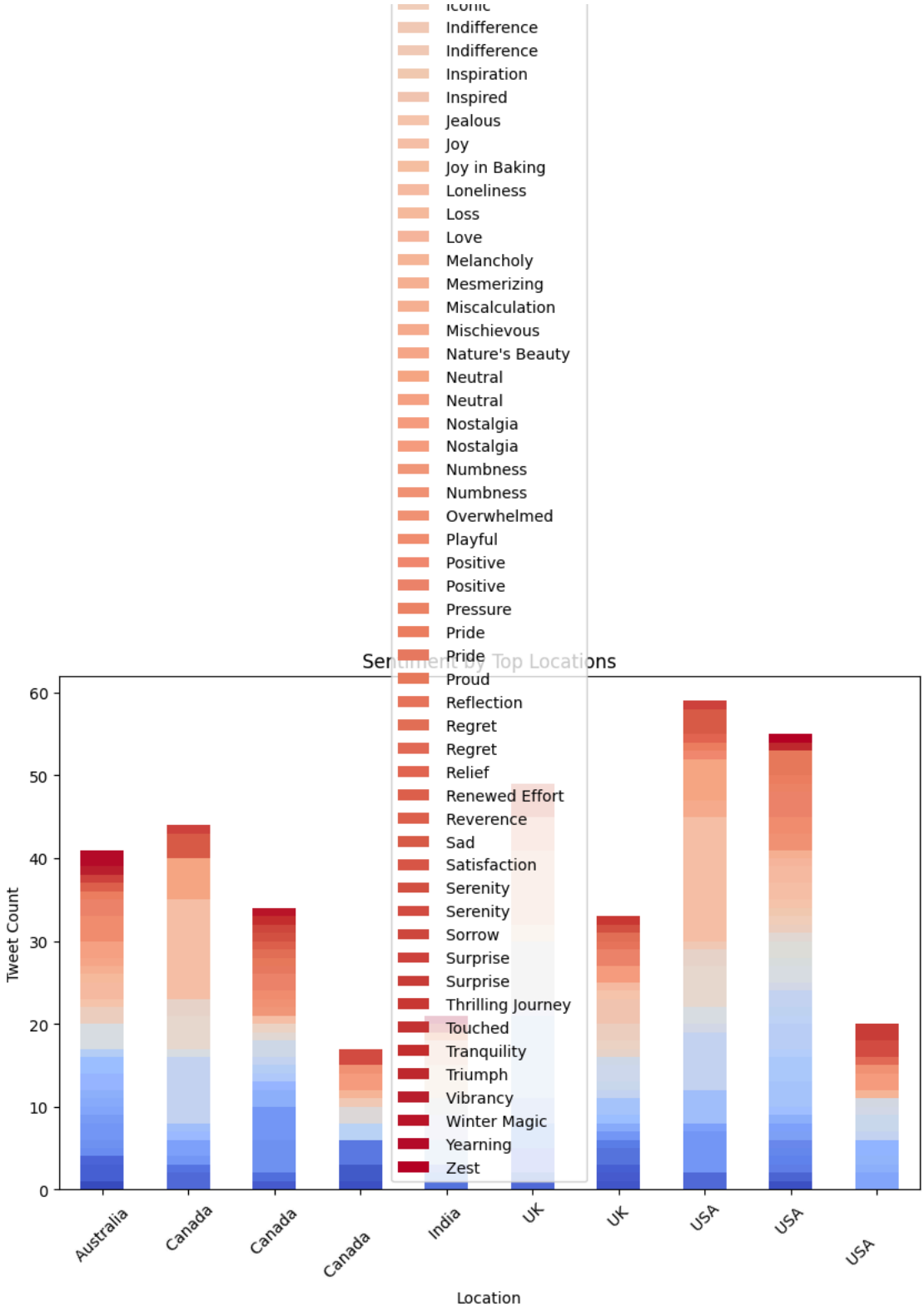



```
top_locations.plot(kind='pie', autopct='%1.1f%%', startangle=140, figsize=(6,6), colormap='Pastel1')
plt.title("Tweet Share by Top Locations")
plt.ylabel("")
plt.show()
```




```
sentiment_by_location = df[df['Country'].isin(top_location_names)].groupby(['Country', 'Sentiment']).size().unstack().fillna(0)

# Plot using matplotlib
sentiment_by_location.plot(kind='bar', stacked=True, figsize=(10,6), colormap='coolwarm')
plt.title("Sentiment by Top Locations")
plt.xlabel("Location")
plt.ylabel("Tweet Count")
plt.xticks(rotation=45)
plt.legend(title="Sentiment")
plt.tight_layout()
plt.show()
```

```
<ipython-input-28-8ed59046b077>:10: UserWarning: Tight layout not applied. The bottom and top margins cannot be made large enough to
  plt.tight_layout()
```


Indifference
Indifference
Inspiration
Inspired
Jealous
Joy
Joy in Baking
Loneliness
Loss
Love
Melancholy
Mesmerizing
Miscalculation
Mischievous
Nature's Beauty
Neutral
Neutral
Nostalgia
Nostalgia
Numbness
Numbness
Overwhelmed
Playful
Positive
Positive
Pressure
Pride
Pride
Proud
Reflection
Regret
Regret
Relief
Renewed Effort
Reverence
Sad
Satisfaction
Serenity
Serenity
Sorrow
Surprise
Surprise
Thrilling Journey
Touched
Tranquility
Triumph
Vibrancy
Winter Magic
Yearning
Zest
Sentiment by Top Locations
Tweet Count
0
10
20
30
40
50
60
Australia
Canada
Canada
Canada
India
UK
UK
USA
USA
USA
Location

3. Trend analysis : Analyze the time data associated with social media and analyse its trends.

```
import pandas as pd
import matplotlib.pyplot as plt
```

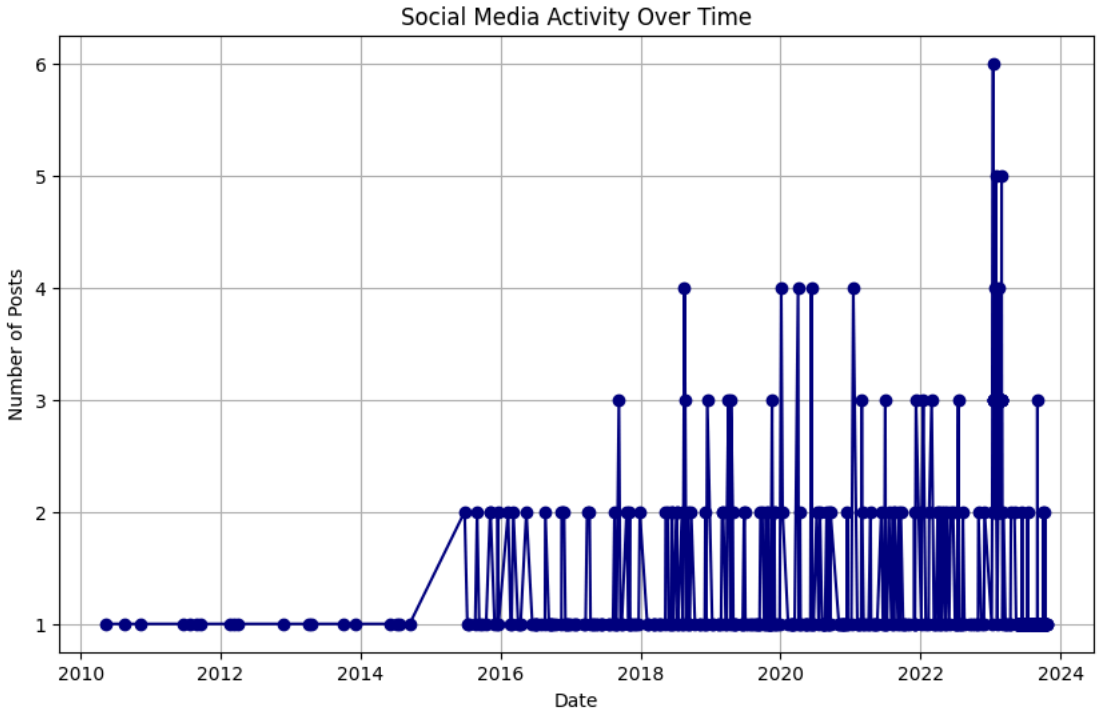
```
df = pd.read_csv('location.csv')
df.head()
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweets	Likes	Country	Year	Month	Day	H
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15.0	30.0	USA	2023	1	15	
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5.0	10.0	Canada	2023	1	15	
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20.0	40.0	USA	2023	1	15	
3	3	3	Excited about the upcoming weekend	Positive	2023-01-15	AdventureX	Facebook	#Travel #Adventure	8.0	15.0	UK	2023	1	15	

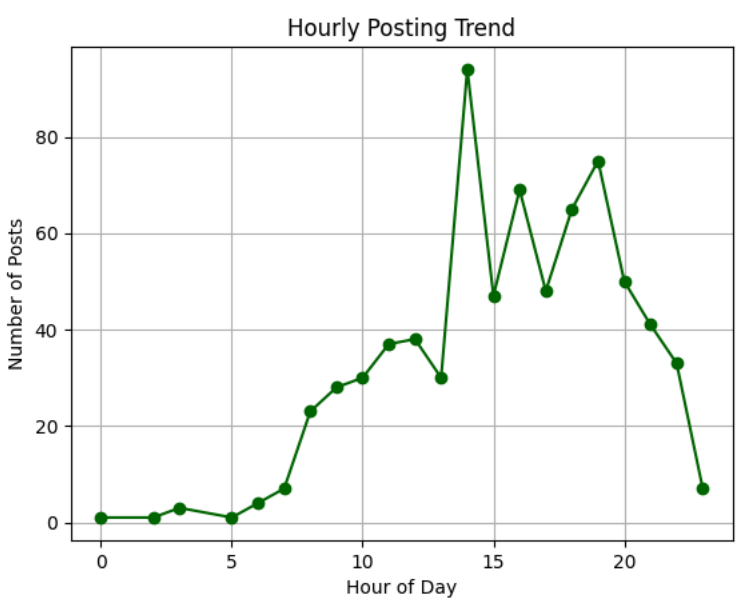
Next steps: Generate code with df | View recommended plots | New interactive sheet

```
df['Timestamp'] = pd.to_datetime(df['Timestamp'])
df['Date'] = df['Timestamp'].dt.date
df['Hour'] = df['Timestamp'].dt.hour
df['Month'] = df['Timestamp'].dt.month
df['Day'] = df['Timestamp'].dt.day
df['Year'] = df['Timestamp'].dt.year
```

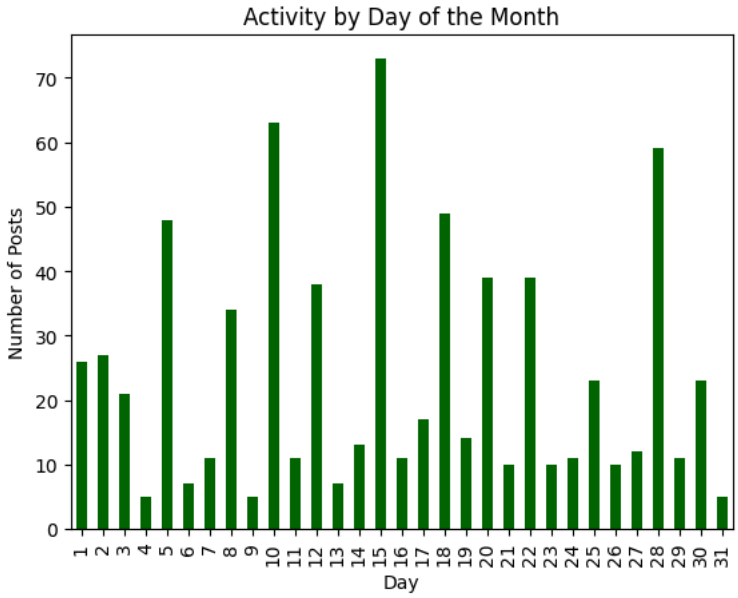
```
trend_by_date = df.groupby('Date').size()
trend_by_date.plot(figsize=(10, 6), marker='o', color='navy')
plt.title("Social Media Activity Over Time")
plt.xlabel("Date")
plt.ylabel("Number of Posts")
plt.grid(True)
plt.show()
```



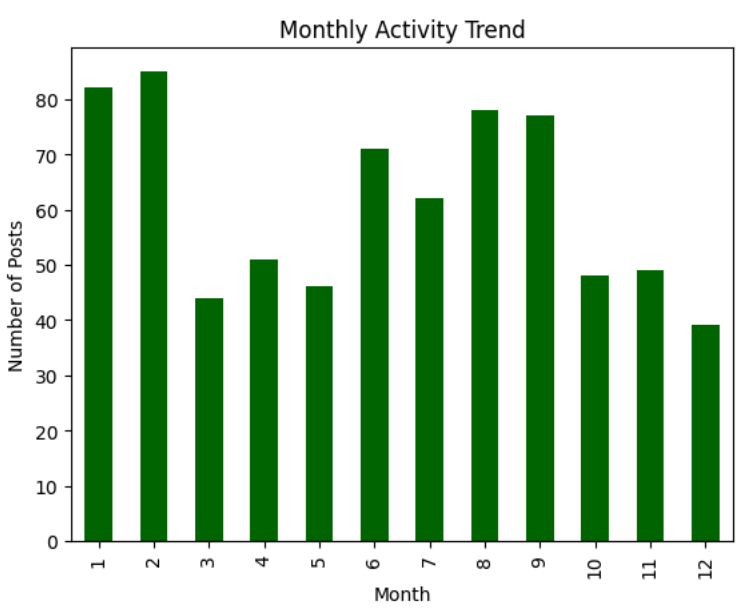

```
hourly_trend = df['Hour'].value_counts().sort_index()
hourly_trend.plot(kind='line', marker='o', color='darkgreen')
plt.title("Hourly Posting Trend")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Posts")
plt.grid(True)
plt.show()
```




```
daily_trend = df['Day'].value_counts().sort_index()
daily_trend.plot(kind='bar', color='darkgreen')
plt.title("Activity by Day of the Month")
plt.xlabel("Day")
plt.ylabel("Number of Posts")
plt.show()
```

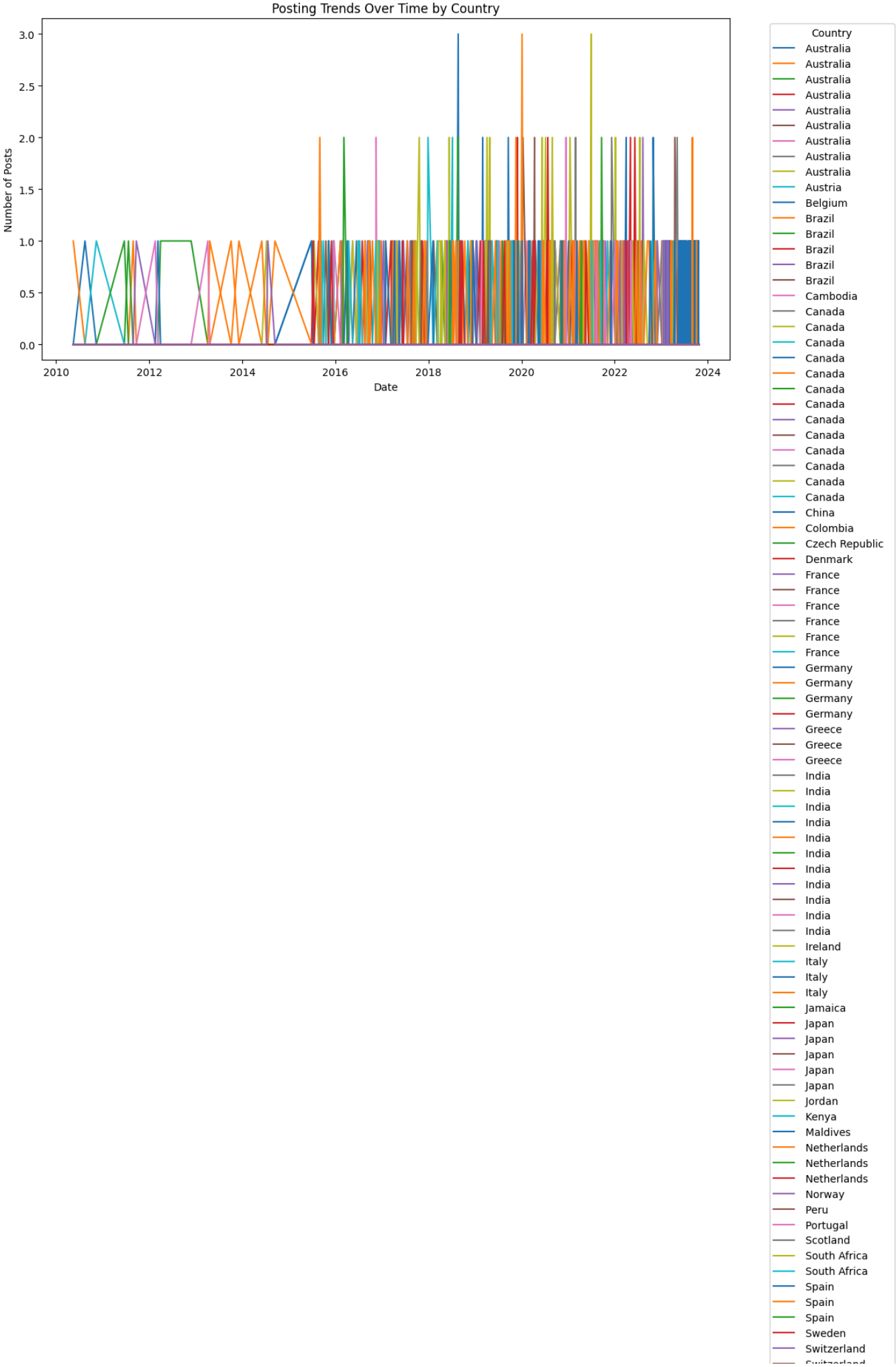



```
monthly_trend = df['Month'].value_counts().sort_index()
monthly_trend.plot(kind='bar', color='darkgreen')
plt.title("Monthly Activity Trend")
plt.xlabel("Month")
plt.ylabel("Number of Posts")
plt.show()
```




```
country_time_trend = df.groupby(['Date', 'Country']).size().unstack(fill_value=0)

country_time_trend.plot(figsize=(12, 6))
plt.title("Posting Trends Over Time by Country")
plt.xlabel("Date")
plt.ylabel("Number of Posts")
plt.legend(title='Country', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

```
<ipython-input-20-2b8328b98015>:8: UserWarning: Tight layout not applied. The bottom and top margins cannot be made large enough to
  plt.tight_layout()
```



4. Hashtag popularity analysis: Determine which hashtags are most popular among different user groups.

```
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
import re
```

```
df = pd.read_csv('location.csv')
df.head()
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweets	Likes	Country	Year	Month	Day	H
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15.0	30.0	USA	2023	1	15	
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5.0	10.0	Canada	2023	1	15	
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20.0	40.0	USA	2023	1	15	
3	3	3	Excited about the upcoming weekend	Positive	2023-01-15	AdventureX	Facebook	#Travel #Adventure	8.0	15.0	UK	2023	1	15	

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
tags = []
for text in df['Hashtags']:
  tags.extend(re.findall(r'#[a-z|A-Z]*', text))
print(tags)
```

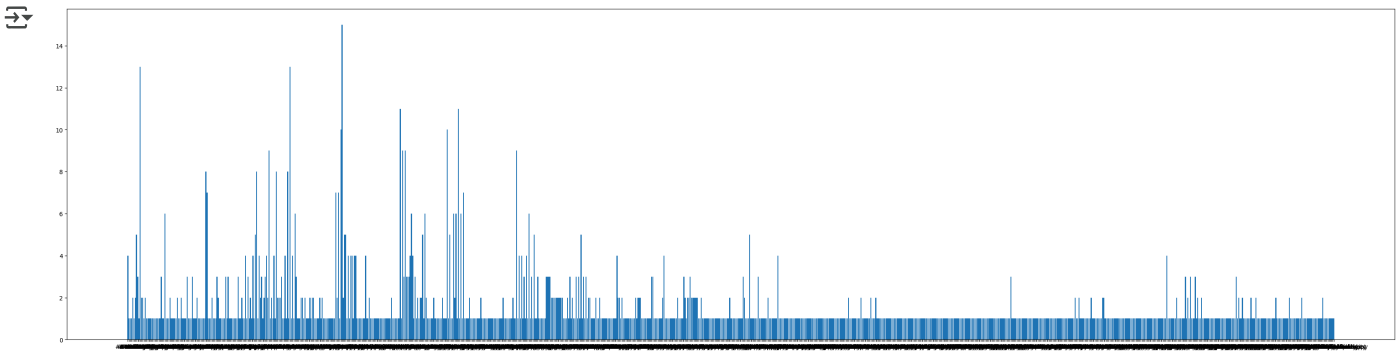
```
['#Nature', '#Park', '#Traffic', '#Morning', '#Fitness', '#Workout', '#Travel', '#Adventure', '#Cooking', '#Food', '#Gratitude', '#F
```

```
element_count = Counter(tags)
print(element_count)
```

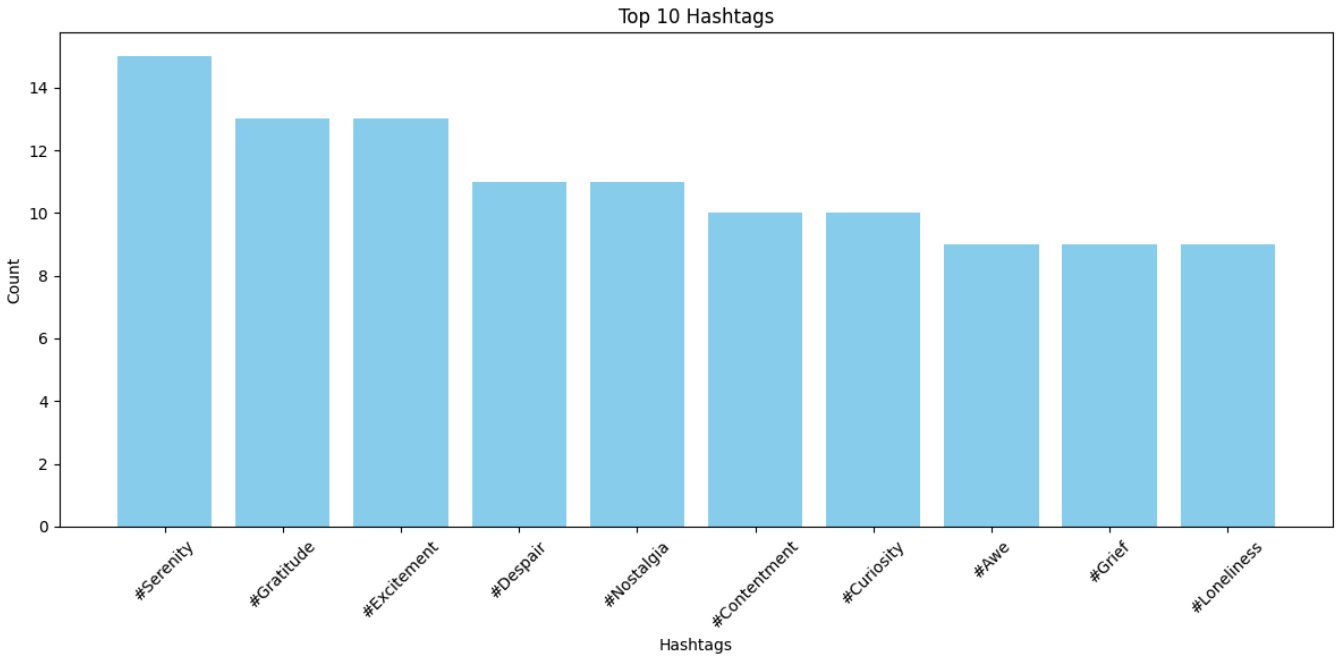
```
Counter({'#Serenity': 15, '#Gratitude': 13, '#Excitement': 13, '#Despair': 11, '#Nostalgia': 11, '#Contentment': 10, '#Curiosity': 1
```

```
counts = []
elements = []
for x in element_count:
    counts.append(element_count.get(x))
    elements.append(x)
```

```
plt.figure(figsize=(40,10))
plt.bar(elements, counts)
plt.show()
```



```
top_hashtags = element_count.most_common(10)
tags, counts = zip(*top_hashtags)

plt.figure(figsize=(12,6))
plt.bar(tags, counts, color='skyblue')
plt.title("Top 10 Hashtags")
plt.xlabel("Hashtags")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



5 Sentiment Analysis for given dataset • Negative Sentiment analysis • Positive Sentiment analysis

```
import pandas as pd
from textblob import TextBlob
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv('location.csv')
df.head()
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweets	Likes	Country	Year	Month	Day	H
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15.0	30.0	USA	2023	1	15	
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5.0	10.0	Canada	2023	1	15	
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20.0	40.0	USA	2023	1	15	
3	3	3	Excited about the upcoming weekend	Positive	2023-01-15	AdventureX	Facebook	#Travel #Adventure	8.0	15.0	UK	2023	1	15	

Next steps: Generate code with df View recommended plots New interactive sheet

```
df['polarity'] = df['Text'].apply(lambda x: TextBlob(x).sentiment.polarity)
df.head()
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweets	Likes	Country	Year	Month	Day	H
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15.0	30.0	USA	2023	1	15	
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5.0	10.0	Canada	2023	1	15	
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20.0	40.0	USA	2023	1	15	
3	3	3	Excited about the upcoming weekend getaway! ...	Positive	2023-01-15 18:20:00	AdventureX	Facebook	#Travel #Adventure	8.0	15.0	UK	2023	1	15	
4	4	4	Trying out a new recipe for dinner tonight. ...	Neutral	2023-01-15 19:55:00	ChefCook	Instagram	#Cooking #Food	12.0	25.0	Australia	2023	1	15	

Next steps: Generate code with df View recommended plots New interactive sheet

```
ans = []
for i in df['polarity']:
  if i > 0.5:
    ans.append('positive')
  else:
    ans.append('negative')

df['pred'] = ans
```

```
df.head()
```

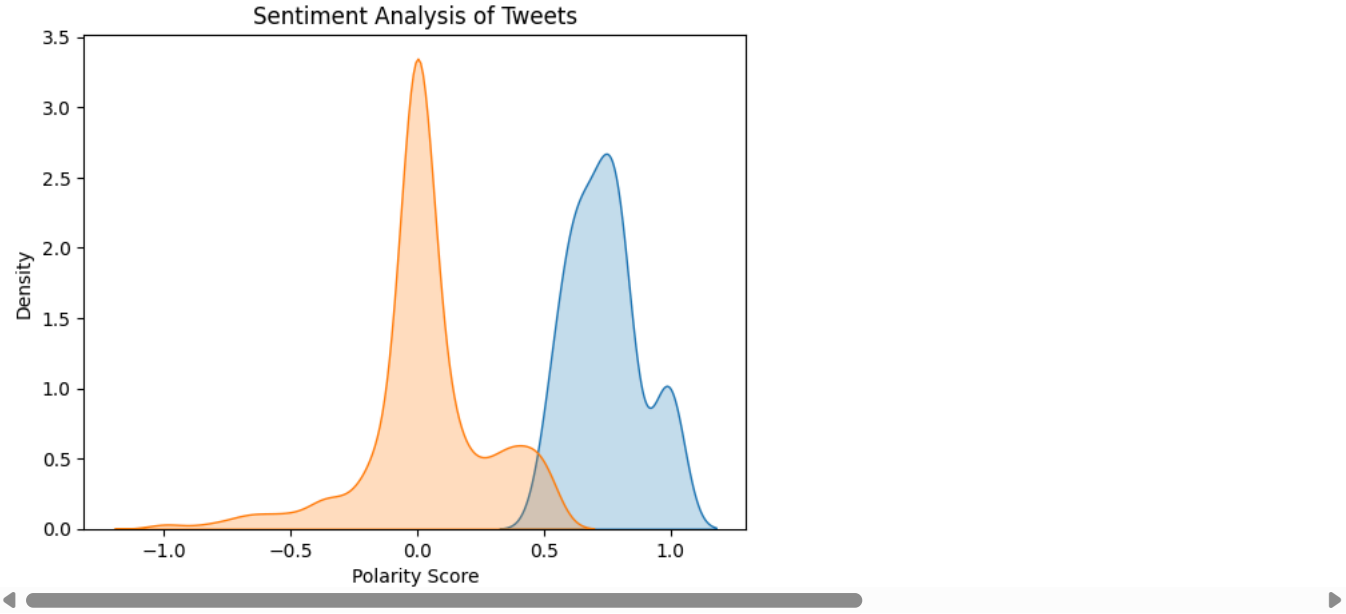
	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweets	Likes	Country	Year	Month	Day	H
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15.0	30.0	USA	2023	1	15	
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5.0	10.0	Canada	2023	1	15	
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20.0	40.0	USA	2023	1	15	
3	3	3	Excited about the upcoming weekend getaway! ...	Positive	2023-01-15 18:20:00	AdventureX	Facebook	#Travel #Adventure	8.0	15.0	UK	2023	1	15	
4	4	4	Trying out a new recipe for dinner tonight. ...	Neutral	2023-01-15 19:55:00	ChefCook	Instagram	#Cooking #Food	12.0	25.0	Australia	2023	1	15	

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
pos_tweets = df[df['pred'] == 'positive']
neg_tweets = df[df['pred'] == 'negative']
```

```
sns.kdeplot(pos_tweets['polarity'], label='Positive', shade=True)
sns.kdeplot(neg_tweets['polarity'], label='negative', shade=True)
plt.xlabel("Polarity Score")
plt.ylabel("Density")
plt.title("Sentiment Analysis of Tweets")
plt.show()
```

```
<ipython-input-16-07139124ef25>:1: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(pos_tweets['polarity'], label='Positive', shade=True)
<ipython-input-16-07139124ef25>:2: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(neg_tweets['polarity'], label='negative', shade=True)
```



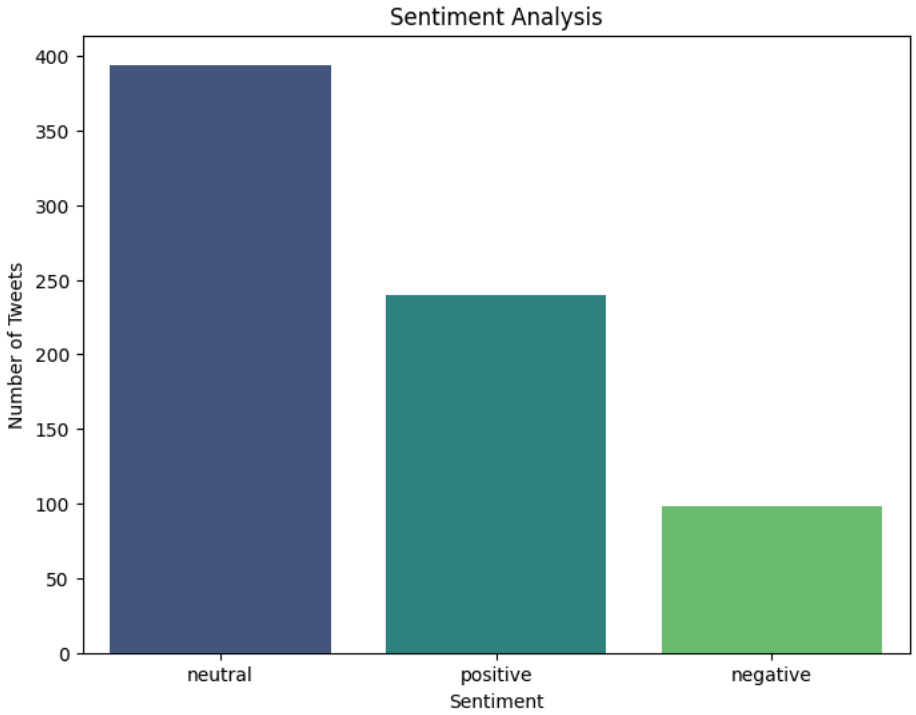

```
ans = []

for i in df["polarity"]:
  if i>0.1:
    ans.append("positive")
  elif(i<-0.1):
    ans.append("negative")
  else:
    ans.append("neutral")

df["sentiment_new"] = ans
```

```
sentiment_counts = df['sentiment_new'].value_counts()
print(sentiment_counts)
```

```
sentiment_new
neutral     394
positive    240
negative     98
Name: count, dtype: int64
```

```
plt.figure(figsize=(8, 6))
sns.barplot(x=sentiment_counts.index, y=sentiment_counts.values, palette="viridis")
plt.xlabel('Sentiment')
plt.ylabel('Number of Tweets')
plt.title('Sentiment Analysis')
plt.show()
```

```
<ipython-input-20-80a1c75b7baa>:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `le

  sns.barplot(x=sentiment_counts.index, y=sentiment_counts.values, palette="viridis")
```




```
df["pred"]
```

	pred
0	positive
1	negative
2	positive
3	negative
4	negative
...	...
727	positive
728	positive
729	positive
730	positive
731	positive

732 rows × 1 columns

dtype: object

```
pos_tweets = df[df["sentiment_new"] == "positive"]
neg_tweets = df[df["sentiment_new"] == "negative"]
neutral_tweets = df[df["sentiment_new"] == "neutral"]
```

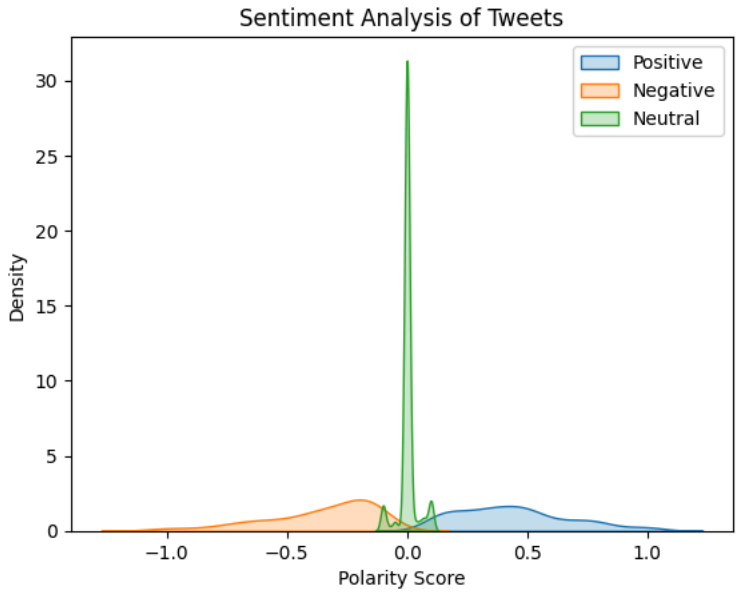
```
print("Positive Tweets:")
for tweet in pos_tweets['Text'].head():
  print("-", tweet)

print("\nNegative Tweets:")
for tweet in neg_tweets['Text'].head():
  print("-", tweet)

print("\nNeutral Tweets:")
for tweet in neutral_tweets['Text'].head():
  print("-", tweet)
```

```
Positive Tweets:
-  Enjoying a beautiful day at the park!
```

```
- Just finished an amazing workout! 💪
- Excited about the upcoming weekend getaway!
- Trying out a new recipe for dinner tonight.
- The new movie release is a must-watch!

Negative Tweets:
- Traffic was terrible this morning.
- Feeling grateful for the little things in life.
- Missing summer vibes and beach days.
- Exploring the city's hidden gems.
- Reflecting on the past and looking ahead.

Neutral Tweets:
- Rainy days call for cozy blankets and hot cocoa.
- Political discussions heating up on the timeline.
- Feeling a bit under the weather today.
- Late-night gaming session with friends.
- Attending a virtual conference on AI.
```

```
sns.kdeplot(pos_tweets["polarity"], shade=True, label="Positive")
sns.kdeplot(neg_tweets["polarity"], shade=True, label="Negative")
sns.kdeplot(neutral_tweets["polarity"], shade=True, label="Neutral")
plt.xlabel("Polarity Score")
plt.ylabel("Density")
plt.title("Sentiment Analysis of Tweets")
plt.legend()
plt.show()
```

```
<ipython-input-24-ff68fa26d5ad>:1: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(pos_tweets["polarity"], shade=True, label="Positive")
<ipython-input-24-ff68fa26d5ad>:2: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(neg_tweets["polarity"], shade=True, label="Negative")
<ipython-input-24-ff68fa26d5ad>:3: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

  sns.kdeplot(neutral_tweets["polarity"], shade=True, label="Neutral")
```




```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

6 User engagement analysis: analyze how users engage with content on social media to understand what types of content are most engaging

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from textblob import TextBlob
from wordcloud import WordCloud
```

```
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
df = pd.read_csv('location.csv')
df.head()
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweets	Likes	Country	Year	Month	Day	H
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15.0	30.0	USA	2023	1	15	
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5.0	10.0	Canada	2023	1	15	
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20.0	40.0	USA	2023	1	15	
3	3	3	Excited about the upcoming weekend	Positive	2023-01-15	AdventureX	Facebook	#Travel #Adventure	8.0	15.0	UK	2023	1	15	

Next steps: Generate code with df | View recommended plots | New interactive sheet

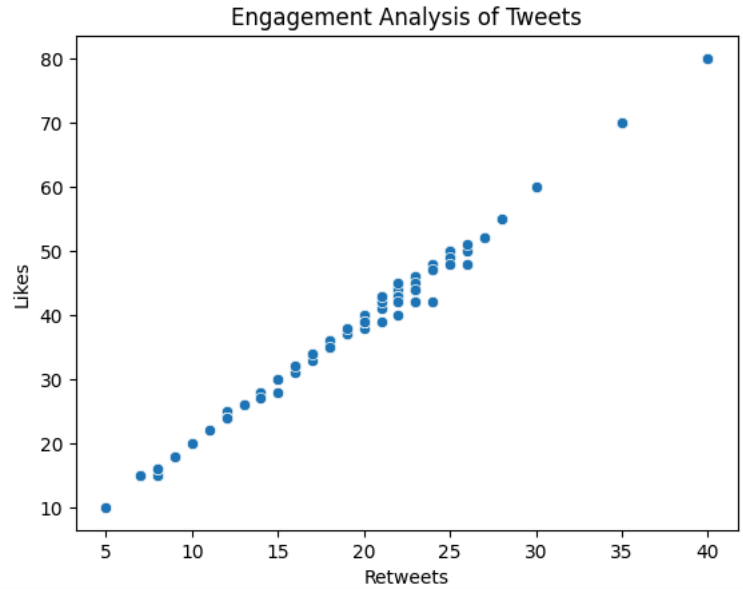
```
stop_words = set(stopwords.words('english'))
df["Text"] = df["Text"].apply(lambda x: " ".join(word for word in x.split() if word.lower() not in stop_words))
```

```
df["polarity"] = df["Text"].apply(lambda x: TextBlob(x).sentiment.polarity)
df.head()
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweets	Likes	Country	Year	Month	Day	H
0	0	0	Enjoying beautiful day park!	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15.0	30.0	USA	2023	1	15	
1	1	1	Traffic terrible morning.	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5.0	10.0	Canada	2023	1	15	
2	2	2	finished amazing workout! 💪	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20.0	40.0	USA	2023	1	15	
3	3	3	Excited upcoming weekend getaway!	Positive	2023-01-15 18:20:00	AdventureX	Facebook	#Travel #Adventure	8.0	15.0	UK	2023	1	15	
4	4	4	Trying new recipe dinner tonight.	Neutral	2023-01-15 19:55:00	ChefCook	Instagram	#Cooking #Food	12.0	25.0	Australia	2023	1	15	

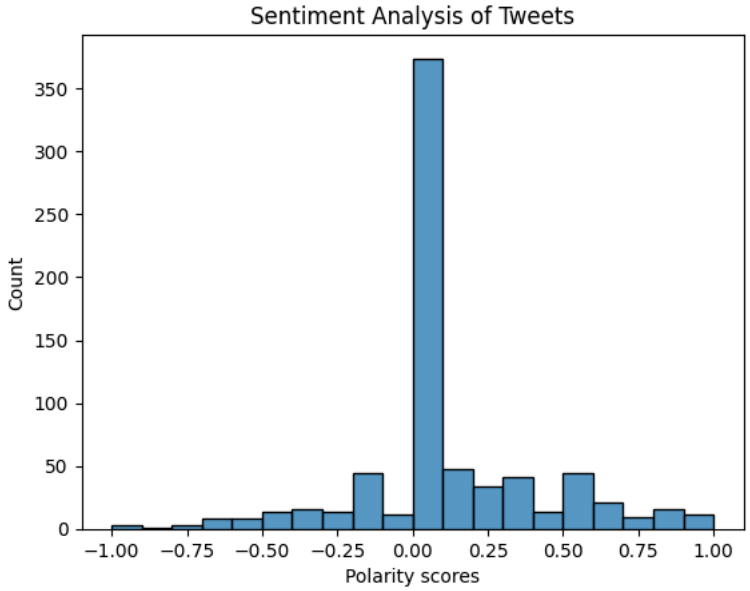
Next steps: Generate code with df | View recommended plots | New interactive sheet

```
sns.scatterplot(x="Retweets", y="Likes", data=df)
plt.xlabel("Retweets")
plt.ylabel("Likes")
plt.title("Engagement Analysis of Tweets")
plt.show()
```




```
wordcloud = WordCloud(background_color="white", max_words=50, contour_width=3, contour_color="steelblue")
wordcloud.generate(" ".join(df["Text"]))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.show()
```

```
sns.histplot(data=df, x=df["polarity"], bins=20)
plt.xlabel("Polarity scores")
plt.ylabel("Count")
plt.title("Sentiment Analysis of Tweets")
plt.show()
```



Exploratory Data Analysis and visualization for given data set

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from wordcloud import WordCloud
import seaborn as sns
```

```
df = pd.read_csv('location.csv')
```

```
print("First 5 rows of datasets: \n")
df.head()
```

```
First 5 rows of datasets:
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtags	Retweets	Likes	Country	Year	Month	Day	H
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Nature #Park	15.0	30.0	USA	2023	1	15	
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffic #Morning	5.0	10.0	Canada	2023	1	15	
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitness #Workout	20.0	40.0	USA	2023	1	15	
3	3	3	Excited about the upcoming weekend	Positive	2023-01-15	AdventureX	Facebook	#Travel #Adventure	8.0	15.0	UK	2023	1	15	

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
print("\nData Info:\n")
print(df.info())
```

```
Data Info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 732 entries, 0 to 731
Data columns (total 15 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Unnamed: 0.1   732 non-null    int64
 1   Unnamed: 0     732 non-null    int64
 2   Text           732 non-null    object
 3   Sentiment      732 non-null    object
 4   Timestamp      732 non-null    object
 5   User           732 non-null    object
 6   Platform       732 non-null    object
 7   Hashtags       732 non-null    object
 8   Retweets       732 non-null    float64
 9   Likes          732 non-null    float64
 10  Country        732 non-null    object
 11  Year           732 non-null    int64
 12  Month          732 non-null    int64
 13  Day            732 non-null    int64
 14  Hour           732 non-null    int64
dtypes: float64(2), int64(6), object(7)
memory usage: 85.9+ KB
None
```

```
print("Summary of Dataset: \n")
df.describe()
```

Summary of Dataset:

	Unnamed: 0.1	Unnamed: 0	Retweets	Likes	Year	Month	Day	Hour
count	732.000000	732.000000	732.000000	732.000000	732.000000	732.000000	732.000000	732.000000
mean	366.464481	369.740437	21.508197	42.901639	2020.471311	6.122951	15.497268	15.521858
std	211.513936	212.428936	7.061286	14.089848	2.802285	3.411763	8.474553	4.113414
min	0.000000	0.000000	5.000000	10.000000	2010.000000	1.000000	1.000000	0.000000
25%	183.750000	185.750000	17.750000	34.750000	2019.000000	3.000000	9.000000	13.000000
50%	366.500000	370.500000	22.000000	43.000000	2021.000000	6.000000	15.000000	16.000000
75%	549.250000	553.250000	25.000000	50.000000	2023.000000	9.000000	22.000000	19.000000
max	732.000000	736.000000	40.000000	80.000000	2023.000000	12.000000	31.000000	23.000000

```
print("Data Types of columns: \n")
print(df.dtypes)
```

Data Types of columns:

```
Unnamed: 0.1      int64
Unnamed: 0        int64
Text             object
Sentiment        object
Timestamp        object
User             object
Platform         object
Hashtags         object
Retweets        float64
Likes           float64
Country          object
Year              int64
Month             int64
Day               int64
Hour              int64
dtype: object
```

```
print("Missing values (if any): \n")
print(df.isnull().sum())
```

Missing values (if any):

```
Unnamed: 0.1    0
Unnamed: 0      0
Text            0
Sentiment       0
Timestamp       0
User            0
Platform        0
Hashtags        0
Retweets        0
Likes           0
Country         0
Year            0
Month           0
Day             0
Hour            0
dtype: int64
```

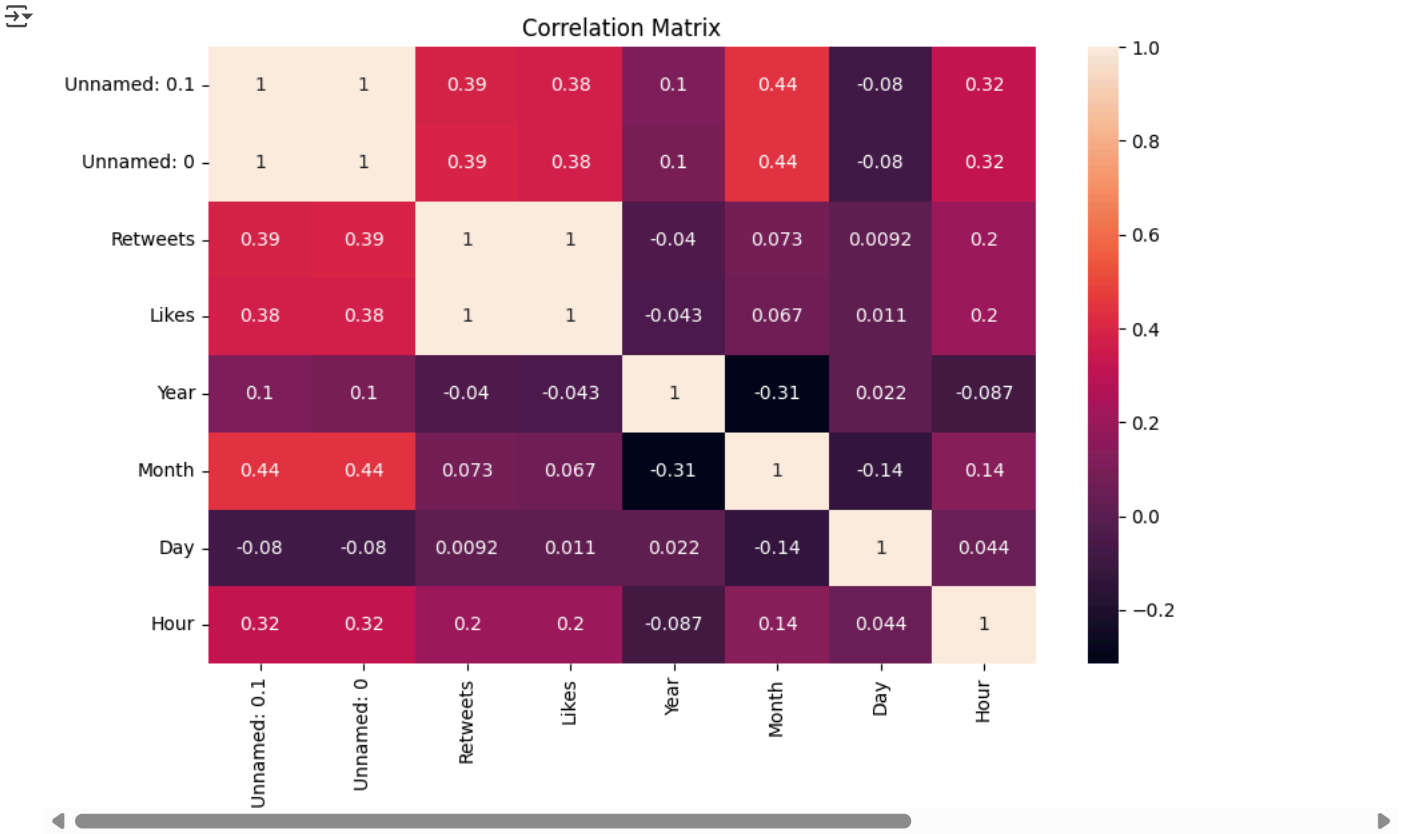
```
numeric_cols = df.select_dtypes(['int64', 'float64']).columns
corr_matrix = df[numeric_cols].corr()

print("Correlation matrix: \n")
print(corr_matrix)
```

Correlation matrix:

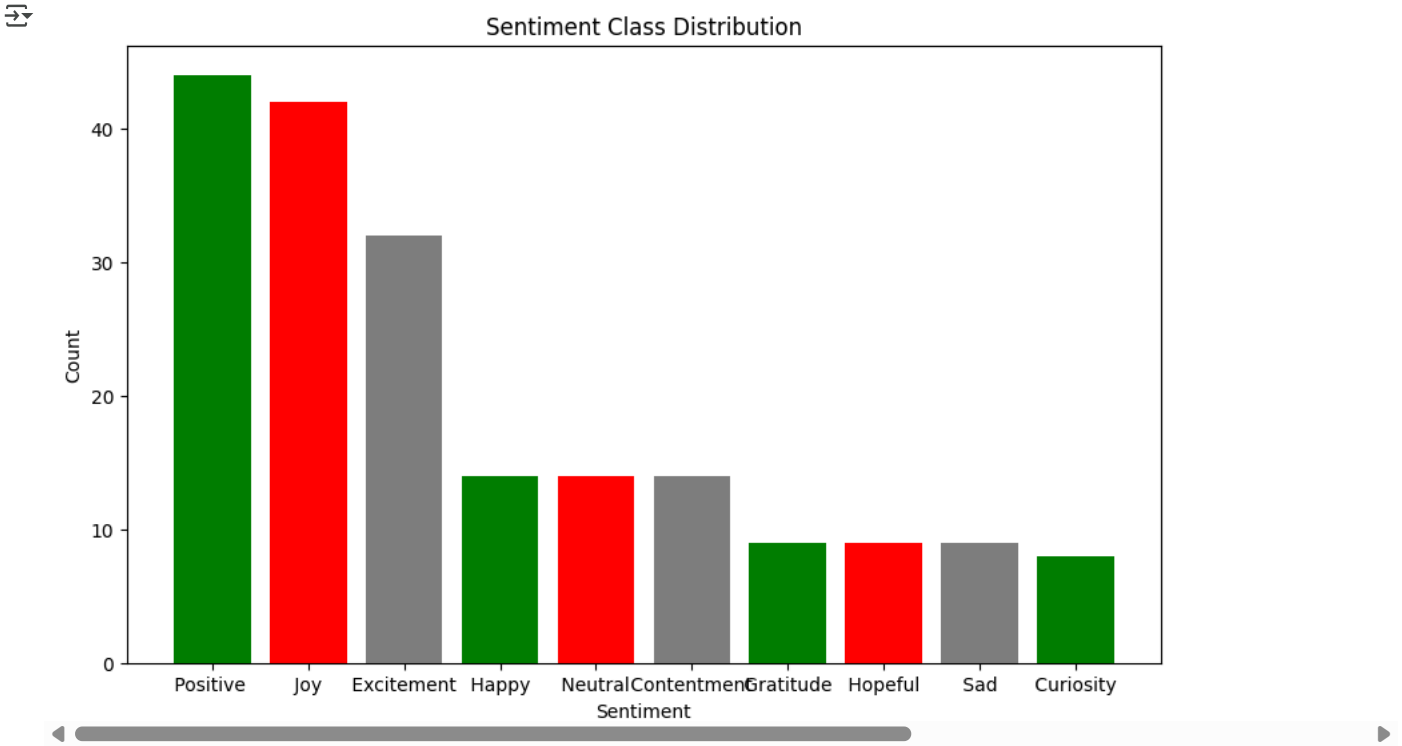
```
              Unnamed: 0.1  Unnamed: 0  Retweets     Likes      Year  \
Unnamed: 0.1      1.000000    0.999995  0.388637  0.376208  0.101578
Unnamed: 0        0.999995    1.000000  0.388884  0.376472  0.100749
Retweets          0.388637    0.388884  1.000000  0.998482 -0.039982
Likes             0.376208    0.376472  0.998482  1.000000 -0.043415
Year              0.101578    0.100749 -0.039982 -0.043415  1.000000
Month             0.443013    0.443523  0.073265  0.066643 -0.314845
Day              -0.080101   -0.080480  0.009213  0.011489  0.021973
Hour              0.322371    0.322163  0.196955  0.195331 -0.087470

                  Month       Day      Hour
Unnamed: 0.1  0.443013 -0.080101  0.322371
Unnamed: 0    0.443523 -0.080480  0.322163
Retweets      0.073265  0.009213  0.196955
Likes         0.066643  0.011489  0.195331
```

```
Year         -0.314845  0.021973 -0.087470
Month         1.000000 -0.135873  0.137835
Day          -0.135873  1.000000  0.044072
Hour          0.137835  0.044072  1.000000
```

```
plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix, annot=True)
plt.title("Correlation Matrix")
plt.show()
```

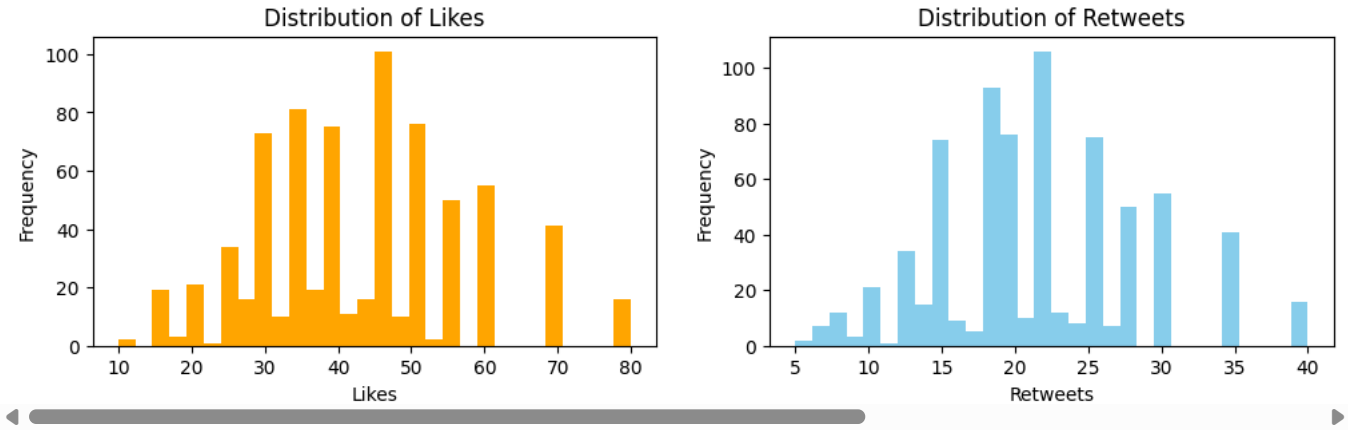



```
sentiment_ctr = df['Sentiment'].value_counts().head(10)
plt.figure(figsize=(10, 6))
plt.bar(sentiment_ctr.index, sentiment_ctr.values, color=['green', 'red', 'gray'])
plt.title('Sentiment Class Distribution')
plt.xlabel('Sentiment')
plt.ylabel('Count')
plt.show()
```

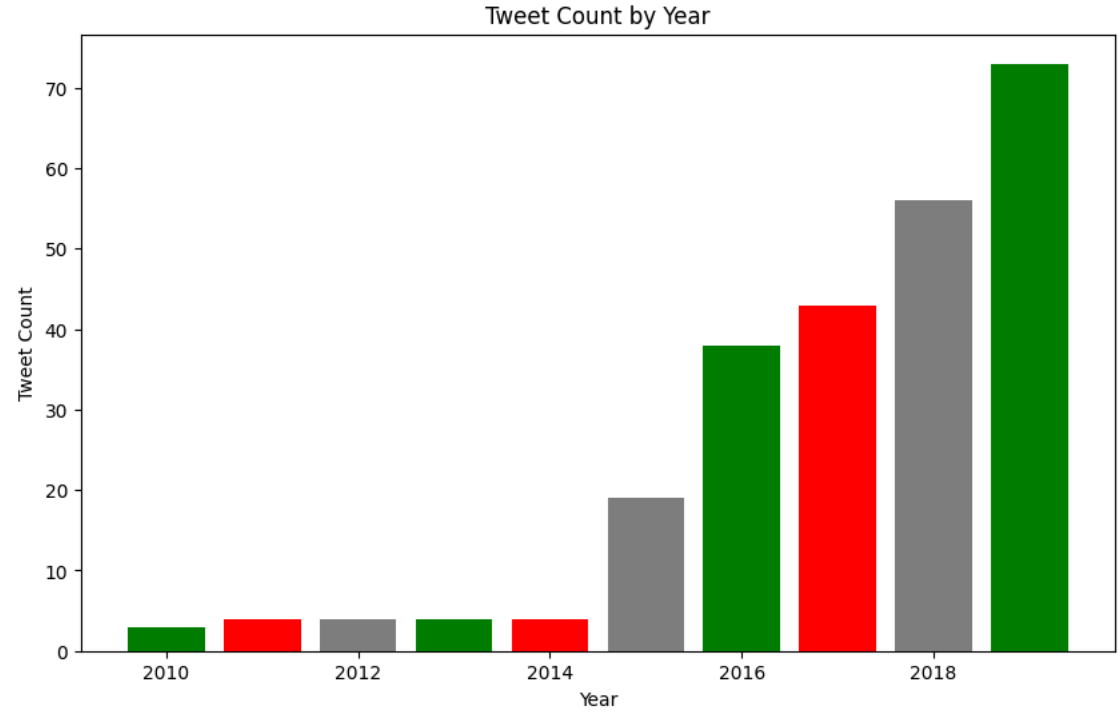


```
plt.figure(figsize=(12 ,3))

plt.subplot(1, 2, 1)
plt.hist(df['Likes'].dropna(), color=['orange'], bins=30)
plt.title('Distribution of Likes')
plt.xlabel('Likes')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
plt.hist(df['Retweets'].dropna(), color=['skyblue'], bins=30)
plt.title('Distribution of Retweets')
plt.xlabel('Retweets')
plt.ylabel('Frequency')
```

```
Text(0, 0.5, 'Frequency')
```




```
year_ctr = df['Year'].value_counts().sort_index().head(10)
plt.figure(figsize=(10, 6))
plt.bar(year_ctr.index, year_ctr.values, color=['green', 'red', 'gray'])
plt.title('Tweet Count by Year')
plt.xlabel('Year')
plt.ylabel('Tweet Count')
plt.show()
```




```
all_text = ' '.join(df['Text'].dropna().tolist())
wordcloud = WordCloud(width=1000, height=500, background_color='white').generate(all_text)

plt.figure(figsize=(15, 7))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("Word Cloud of Tweet Texts")
plt.show()
```

Word Cloud of Tweet Texts

8. Brand analysis: analyze the conversation around a particular brand for given dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from wordcloud import WordCloud
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.sentiment import SentimentIntensityAnalyzer
from textblob import TextBlob
import re
from collections import Counter
```

```
df = pd.read_csv('location.csv')
df.head()
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtag
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Natur #Par
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffi #Mornin
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitnes #Workou
3	3	3	Excited about the upcoming weekend	Positive	2023-01-15	AdventureX	Facebook	#Trave #Adventur

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('vader_lexicon')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
True
```

```
stop_words = set(stopwords.words('english'))
sia = SentimentIntensityAnalyzer()
```

```
def pre_process(text):
    tokens = word_tokenize(text.lower())
    filtered_tokens = [token for token in tokens if token.isalnum() and token not in stop
    return filtered_tokens
```

```
df['tokens'] = df['Text'].apply(pre_process)
df['Sentiment_new'] = df['Text'].apply(lambda x: TextBlob(x).sentiment.polarity)
df.head()
```

	Unnamed: 0.1	Unnamed: 0	Text	Sentiment	Timestamp	User	Platform	Hashtag
0	0	0	Enjoying a beautiful day at the park! ...	Positive	2023-01-15 12:30:00	User123	Twitter	#Natur #Par
1	1	1	Traffic was terrible this morning. ...	Negative	2023-01-15 08:45:00	CommuterX	Twitter	#Traffi #Mornin
2	2	2	Just finished an amazing workout! 💪 ...	Positive	2023-01-15 15:45:00	FitnessFan	Instagram	#Fitnes #Workou
3	3	3	Excited about the upcoming weekend getaway! ...	Positive	2023-01-15 18:20:00	AdventureX	Facebook	#Trave #Adventur
4	4	4	Trying out a new recipe for dinner tonight. ...	Neutral	2023-01-15 19:55:00	ChefCook	Instagram	#Cookin #Foo

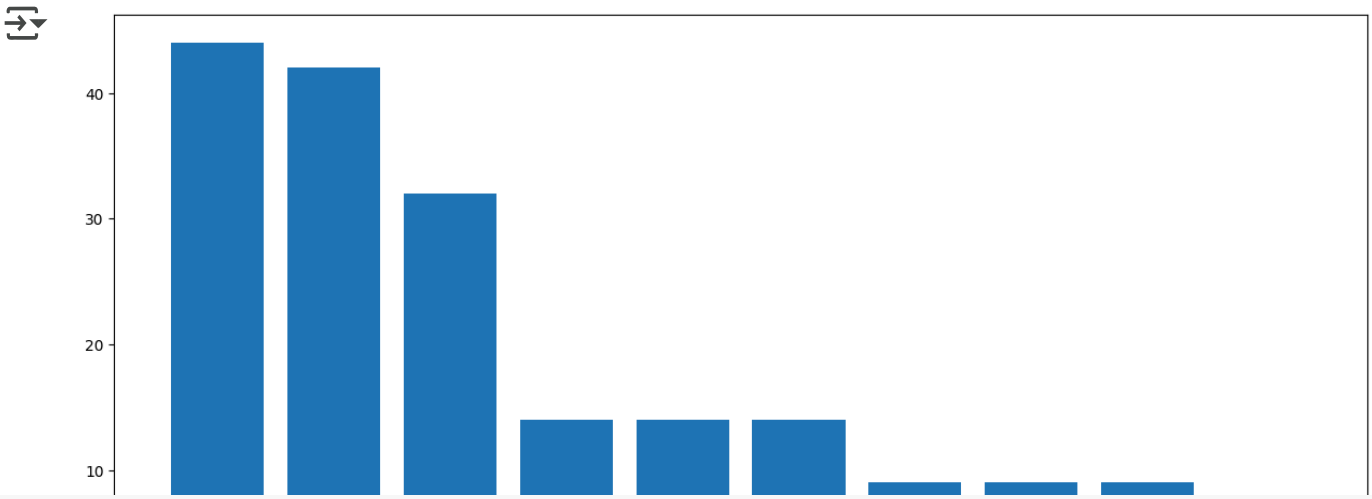
Next steps: Generate code with df | View recommended plots | New interactive sheet

```
all_text = ' '.join(df['Text'])
wordcloud = WordCloud(width=800, height=500, background_color='white').generate(all_text)

plt.figure(figsize=(10, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.title("WordCloud of Brand Mentions")
plt.axis('off')
plt.show()
```

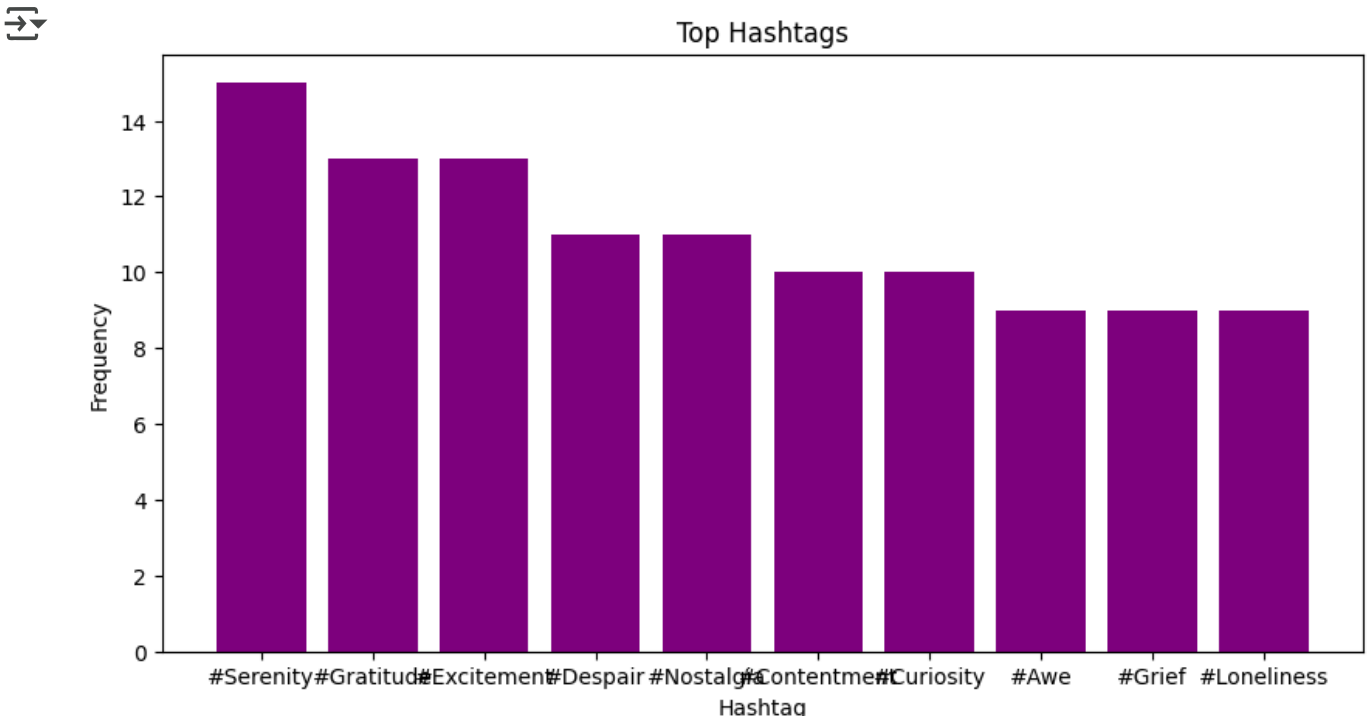

```
average_sentiment = df['Sentiment_new'].mean()
print(f"Average sentiment of brand mentions: {average_sentiment:.2f}")
```

```
Average sentiment of brand mentions: 0.10
```

```
new_df = df['Sentiment'].value_counts().reset_index().head(10)
plt.figure(figsize=(15, 7))
plt.bar(new_df['Sentiment'], new_df['count'])
plt.show()
```




```
hashtags = []
for text in df['Hashtags'].dropna():
    hashtags.extend(re.findall(r'#\w+', text))

hashtag_ctr = Counter(hashtags).most_common(10)
tags, ctr = zip(*hashtag_ctr)

plt.figure(figsize=(10, 5))
plt.bar(tags, ctr, color='purple')
plt.title(f"Top Hashtags")
plt.xlabel('Hashtag')
plt.ylabel('Frequency')
plt.show()
```



```
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
import nltk
import random
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
```

```
nltk.download('stopwords')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
num_posts = 10
words = ['social', 'media', 'competitor', 'activity', 'analysis', 'keyword', 'extraction', 'experiment']

df = pd.DataFrame({
    'post_id': range(1, num_posts + 1),
    'text': [
        f"Post {i}: {' '.join(random.choices(words, k=random.randint(5, 15)))}"
        for i in range(1, num_posts + 1)
    ]
})
```

```
df.head()
```

Show hidden output

```
word_ds = []

stop_words = set(stopwords.words('english'))
```

```
def extract_key_words(text):
  words = word_tokenize(text.lower())
  keywords = [word for word in words if word.isalnum() and word not in stop_words]
  word_ds.extend(keywords)
  return ' '.join(keywords)
```

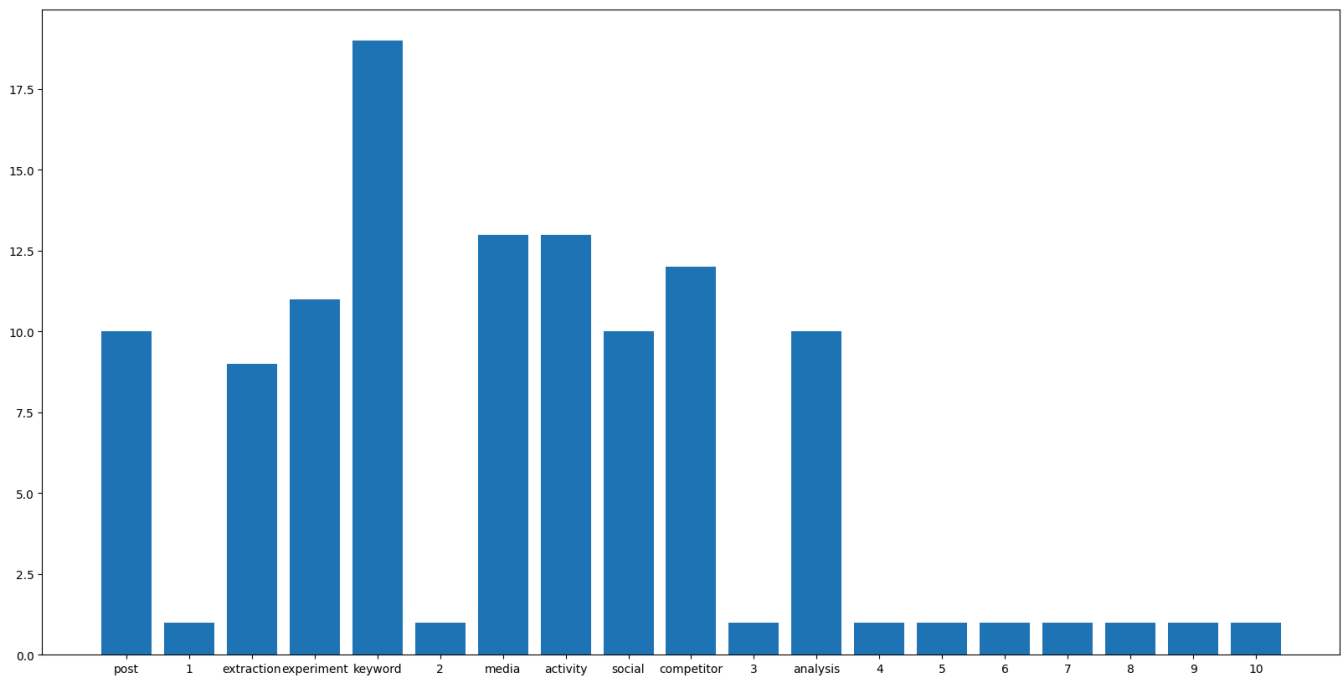
```
df['keywords'] = df['text'].apply(extract_key_words)
df.head()
```

	post_id	text	keywords
0	1	Post 1: extraction experiment extraction keywo...	post 1 extraction experiment extraction keywor...
1	2	Post 2: keyword media activity social social k...	post 2 keyword media activity social social ke...
2	3	Post 3: keyword activity media social activity...	post 3 keyword activity media social activity ...
3	4	Post 4: extraction analysis competitor activit...	post 4 extraction analysis competitor activity...
4	5	Post 5: extraction keyword keyword social extr...	post 5 extraction keyword keyword social extra...

```
counts = dict(Counter(word_ds))
print(counts)
```

```
{'post': 10, '1': 1, 'extraction': 9, 'experiment': 11, 'keyword': 19, '2': 1, 'media': 13, 'activity': 13, 'social': 10, 'competitc
```

```
keys = []
values = []
for key, value in counts.items():
  keys.append(key)
  values.append(value)
plt.figure(figsize=(20,10))
plt.bar(keys, values)
plt.show()
```




```
#run again sentiment anaylysis
```

Social Network data analysis for community detection and influential analytics for given problem using Girvan newman algorithm / Kmean clustering algorithm

```
import pandas as pd
import networkx as nx
from networkx.algorithms import community
from sklearn.cluster import KMeans
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

```
df = pd.read_csv('tweets.csv')
df.head()
```

	created_at	tweet	user	user_location	retweet_count	additional_data
0	05-02-2023 11:37	Anyone who Belongs To The Society,\nWhoever Wi...	PVRTWEETSSS	NaN	0	{'created_at': 'Sun Feb 05 11:37:50 +0000 2023...
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...
2	05-02-2023 10:03	And people call him bad?\n#tate #FreeTheTates ...	hsenshamass	NaN	0	{'created_at': 'Sun Feb 05 10:03:48 +0000 2023...
3	05-02-2023 08:34	How's this poetry or you can call this poem. #...	kshitiz_999	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 08:34:39 +0000 2023...
4	05-02-2023 08:19	Andrew Tate's Best Piece of advice…\n\nUnleash...	TheWolfCeo	NaN	0	{'created_at': 'Sun Feb 05 08:19:04 +0000 2023...

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
df = df[df.user_location.notnull()][:10]
df['hashtags'] = ''
df.head()
```

	created_at	tweet	user	user_location	retweet_count	additional_data	hashtags
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	
3	05-02-2023 08:34	How's this poetry or you can call this poem. #...	kshitiz_999	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 08:34:39 +0000 2023...	
24	04-02-2023 22:06	@Cobratate Calm down kickboxer boy! We have bi...	after_shook	Place(_api=<tweepy.api.API object at 0x000001B...	2	{'created_at': 'Sat Feb 04 22:06:17 +0000 2023...	
31	04-02-2023	@Cobratate I'm anti	after_shook	Place(_api=<tweepy.api.API object	1	{'created_at': 'Sat Feb 04	

```
for i, row in df.iterrows():
  hashtags = [token for token in row.tweet.split() if token.startswith('#')]
  df['hashtags'][i] = hashtags

df.head()
```

Show hidden output

```
df = df.explode('hashtags')
users = list(df['user'].unique())
hashtags = list(df['hashtags'].unique())
df.head()
```

	created_at	tweet	user	user_location	retweet_count	additional_data	hashtags
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#NewProfilePic
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#AndrewTate
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#khabib

```
vis = nx.Graph()
vis.add_nodes_from(users+hashtags)

for name, group in df.groupby(['hashtags', 'user']):
  hashtag, user = name
  weight = len(group)
  vis.add_edge(hashtag, user, weight=weight)
```

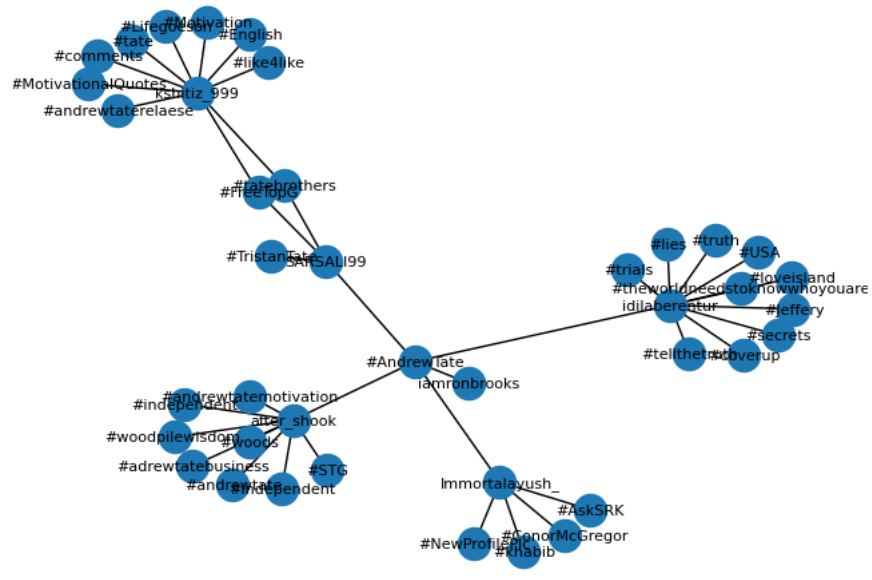
```
community_gen = community.girvan_newman(vis)
top_level_communities = next(community_gen)
print(community_gen)
```

```
<generator object girvan_newman at 0x7f1f3a321300>
```

```
communities = list(next(community_gen))
print(communities)
```

```
[{'#khabib', '#independent', '#woodpilewisdom.', '#AndrewTate', '#Independent', '#andrewtatemotivation', 'Immortalayush_', '#ConorMc
```

```
nx.draw(vis, with_labels=True, font_size=8)
plt.axis("off")
plt.show()
```




```
centrality = nx.betweenness_centrality(vis)
print(centrality)
```

```
{'Immortalayush_': 0.1970310391363023, 'kshitiz_999': 0.3731443994601889, 'after_shook': 0.3724696356275303, 'SARSALI99': 0.4527665
```

```
print('Top 5 Most Influential Users/Hashtags based on betweenness centrality :- ')
influential_nodes = sorted(
    centrality.items(), key= lambda item: item[1], reverse=True
)[:5]
print(centrality.items())
```

```
Top 5 Most Influential Users/Hashtags based on betweenness centrality :-
dict_items([('Immortalayush_', 0.1970310391363023), ('kshitiz_999', 0.3731443994601889), ('after_shook', 0.3724696356275303), ('SARS
```

```
for nodes in influential_nodes:
  print(f"{nodes[0]}: {nodes[1]}")
```

```
#AndrewTate: 0.7584345479082321
SARSALI99: 0.45276653171390013
idilaberentur: 0.45209176788124156
kshitiz_999: 0.3731443994601889
after_shook: 0.3724696356275303
```

```
label_encoder = LabelEncoder()
```

```
df['user_label'] = label_encoder.fit_transform(df['user'])
df['hashtag_label'] = label_encoder.fit_transform(df['hashtags'])
df.head()
```

	created_at	tweet	user	user_location	retweet_count	additional_data	hashtags	user_label	hashtag
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#NewProfilePic	0	
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#AndrewTate	0	
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#khabib	0	
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#ConorMcGregor	0	
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#AskSRK	0	

```
X = df[['user_label', 'hashtag_label']]
X.head()
```

	user_label	hashtag_label
1	0	10
1	0	0
1	0	21
1	0	2
1	0	1

Next steps: Generate code with X | View recommended plots | New interactive sheet

```
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)
df['cluster'] = kmeans.labels_
df.head()
```

	created_at	tweet	user	user_location	retweet_count	additional_data	hashtags	user_label	hashtag
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#NewProfilePic	0	
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#AndrewTate	0	
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#khabib	0	
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#ConorMcGregor	0	
1	05-02-2023 10:51	#NewProfilePic\nKeep Grinding, for the loved o...	Immortalayush_	Place(_api=<tweepy.api.API object at 0x000001B...	0	{'created_at': 'Sun Feb 05 10:51:44 +0000 2023...	#AskSRK	0	

```
plt.scatter(df['user_label'], df['hashtag_label'], c=df['cluster'], cmap='viridis')
plt.xlabel('User Labels')
print('Hashtags Labels')
plt.title('K-Means Clustering')
plt.colorbar(label='Cluster')
plt.show()
```

```
Hashtags Labels
```